Indian Academy of Sciences, Bengaluru
Indian National Science Academy, New Delhi
The National Academy of Sciences India, Prayagraj

SUMMER RESEARCH FELLOWSHIPS — 2025

Format for the final Report*,^

Name of the candidate	:	Aryan Shrivastava
Application Registration no.	:	PHYS364-I
Date of joining	:	02/06/2025
Date of completion	:	28/07/2025
Total no. of days worked	:	56
Name of the guide	:	Dr. Moon Moon Devi
Guide's institution	:	Tezpur University, Tezpur
Project title	:	Particle Physics: Simulations and Machine Learning

Address with pin code to which the certificate could be sent:

Krishna Hostel, NISER, Jatni, Khurdha
Odisha - 752050

E-mail ID: aryan.shrivastava@niser.ac.in

Phone No: 9425168926

TA Form attached with final report : YES ____ NO ✓

If, NO, Please specify reason: Claimed through INSPIRE

Aryan S.
Signature of the candidate

Date: 29/07/2025

Dr. Moon Moon Devi
Assistant Professor
Department of Physics
Tezpur University
Signature of the guide

Date: 29/07/2025

IMPORTANT NOTES:

*** This format should be the first page of the report and should be stapled with the main report. The final report could be anywhere between 20 and 25 pages including tables, figures etc.**

^ The final report must reach the Academy office within 10 days of completion. If delayed fellowship amount will not be disbursed.

(For office use only; do not fill/tear)

Candidate's name:	Fellowship amount:
Student: Teacher:	Deduction:
Guide's name:	TA fare:
KVPY Fellow: INSPIRE Fellow:	Amount to be paid:
PFMS Unique Code:	A/c holder's name:
Others	

Science Academies' Summer Research Fellowship Programme

8-Week Report

Particle Physics: Simulations and Machine Learning

2nd June 2025 - 28 July 2025

Submitted By:

Aryan Shrivastava

2nd Year Integrated M.Sc

School of Physical Sciences

NISER, Bhubaneswar

Project Guide:

Dr. Moon Moon Devi

Assistant Professor

Department of Physics

Tezpur University, Tezpur

DECLARATION

I hereby declare that I am the author of this project report, which was carried out under the guidance of Dr. Moon Moon Devi, Assistant Professor at the Department of Physics, Tezpur University.

Signature of Student:
Date: 28/07/2025

The project report entitled "Particle Physics: Simulations and Machine Learning" was carried out in the Department of Physics, Tezpur University, Tezpur, Assam.

Moon Moon Devi
28/07/2025

Dr. Moon Moon Devi
Assistant Professor
Department of Physics
Tezpur University

Signature of Supervisor:
Date:

Signature of Student:
Date: 28/07/2025.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Moon Moon Devi, for her guidance and support throughout the course of this summer internship. Her insights and expertise have played a crucial role in this project. I am grateful for the opportunity to work under her mentorship.

I sincerely express my heartfelt gratitude to the IASc-INSA-NASI Summer Research Fellowship Programme for providing me with this valuable opportunity to undertake a summer internship.

I would also like to thank the research scholars Mr. Arnab Sarkar, Ms. Dharitree Bezboruah, and Mr. Priyankush Deka for their consistent support and valuable assistance throughout the course of this project.

Lastly, I extend my heartfelt thanks to my brother for his unwavering moral support throughout this journey.

ABSTRACT

Simulations and machine learning have become essential tools in high-energy physics, enabling precise modeling of detector responses and data-driven approaches to event reconstruction. This report explores machine learning applications in the context of the INO-ICAL detector, focusing on muon energy reconstruction as a supervised regression task. Models including Random Forest, XGBoost, and Artificial Neural Networks (ANNs) were implemented, with particular emphasis on the mathematical formulation behind each approach. A comparative analysis revealed that all models performed similarly, with ANNs achieving slightly higher predictive performance. These findings underscore the potential of data-driven techniques for parameter reconstruction in modern particle detectors.

TABLE OF CONTENTS

1 Introduction 5

2 Simulations in Particle Physics 5

2.1 GEANT4 Simulation toolkit 5

2.2 Detector Construction in GEANT4 6

2.3 Rutherford Experiment Simulation 7

3 INO-ICAL Detector 7

3.1 Overall Geometry and Structure 8

3.2 Resistive Plate Chambers 8

3.3 Muon Tracking in ICAL 10

4 Machine Learning 10

4.1 Dataset Description and Preprocessing 11

4.2 Random Forest Regression Model 12

4.3 XGBoost Regression model 15

4.4 Artificial Neural Network (ANN) 17

5 Model Evaluation and Comparison 20

5.1 Standard Evaluation Metrics 20

5.2 Residual distribution evaluation 21

6 Conclusion 23

7 Future Direction 24

1 Introduction

The field of experimental high-energy physics increasingly relies on computational tools to simulate particle interactions and analyze detector data. With the growing complexity of modern detectors and the large volume of data generated in experiments, traditional analysis methods are often insufficient. This has led to the integration of simulation frameworks and machine learning techniques to enhance the design of detectors and interpretation of experimental results.

A major focus of this report is tracker-type detectors, particularly Resistive Plate Chambers (RPCs), which serve as the active elements in the INO-ICAL detector. We explored the use of machine learning models to address the task of muon energy reconstruction from detector data. Using a simulated dataset based on the ICAL geometry, several regression models were implemented, including Artificial Neural Networks (ANNs), Random Forest Regressors, and XGBoost Regressors. These models were trained on features relevant to the muon tracks and evaluated to assess their predictive accuracy.

The objective of this study was to demonstrate the effectiveness of data-driven approaches in improving the precision of energy reconstruction in tracker-type detectors. These techniques can complement traditional statistical methods by providing more refined analyses in experimental high-energy physics.

2 Simulations in Particle Physics

Particle physics experiments aim to probe fundamental interactions at microscopic scales, often under extreme conditions and through rare processes. However, building and operating high-energy detectors is resource-intensive, and not all phenomena can be observed directly because of their statistical rarity and experimental limitations. In this context, simulations are an essential complement to experimental studies.

Simulations provide controlled and configurable conditions for studying particle interactions with matter, detector geometries, and responses. This enables the prediction of outcomes, optimization of detector performance, and precise analysis of event data. For example, by simulating scattering processes, energy depositions, or decay chains, expectations can be established against which actual experimental data can be compared.

The importance of simulations in modern particle physics lies not only in their predictive capabilities but also in their utility for design validation, event reconstruction and uncertainty quantification. Simulated datasets are used to develop reconstruction algorithms, evaluate systematic errors, and benchmark signal extraction techniques. Furthermore, they allow the testing of detector configurations and materials without requiring immediate physical prototyping.

The computational capacity has increased considerably in the recent past. This allows the simulation of complex experimental apparatuses and a wide variety of interactions. Hence, their role continues to expand in the field of science.

2.1 GEANT4 Simulation toolkit

GEANT4[1] is an open-source object-oriented software toolkit designed to simulate the passage of particles through matter. Initially developed to meet the demands of high-energy physics experiments at the Large Hadron Collider (LHC), GEANT4 has evolved

into a versatile platform that is now used across multiple scientific and engineering disciplines, including nuclear physics, medical physics, space science, and radiation protection. Its continued relevance over more than two decades is a result of robust software engineering, extensible architecture, and consistent validation of its physical models against experimental data[2].

The GEANT4 toolkit allows users to construct complete simulation applications by integrating components for geometry definition, material specification, physics process modeling, primary particle generation, event management, and data extraction[3]. This design offers great flexibility and control, enabling detailed simulations of complex detector setups with high physical fidelity.

One of the fundamental features of GEANT4 is its ability to simulate a wide range of physical processes. It includes comprehensive models for electromagnetic interactions (such as ionization, bremsstrahlung, Compton scattering, and pair production), hadronic interactions, particle decay and nuclear processes[4].

The simulation proceeds by generating primary events, tracking the particles through a defined geometry, and invoking relevant physical processes as the particles interact with the matter. These interactions are recorded as hits, steps, and tracks, which can be accessed through sensitive detectors and scorers for data analysis. The Monte Carlo engine governs the probabilistic nature of these interactions using cross-sectional data and random sampling algorithms to ensure statistical realism in the simulations.

Overall, GEANT4 is a cornerstone of modern simulation-based research. Its ability to accurately replicate detector geometries and experimental conditions, along with its computational efficiency and extensive documentation, make it a powerful and indispensable tool in particle physics research.

2.2 Detector Construction in GEANT4

One of the foundational capabilities of the GEANT4 simulation toolkit is its powerful and flexible support for detector design. Defining the geometry and materials of the experimental setup is essential in any simulation of particle interactions with matter. Users define the entire spatial configuration of the experimental apparatus, such as beamlines, targets, tracking detectors, and calorimeters, by specifying a hierarchy of physical volumes composed of logical volumes and geometric solids.

The detector construction in GEANT4 begins with the creation of geometric solids that represent basic shapes such as boxes, cylinders, and spheres. Another way to obtain solids is through a Boolean combination - union, intersection, and subtraction. For instance, Figure 1 illustrates a solid created by subtracting a hemisphere from a cube using the `G4SubtractionSolid` operation. These shapes are defined as logical volumes to which users attribute the material properties. Finally, the physical volumes place these logical volumes into the world coordinate system, applying translations and rotations as needed. It supports nested geometries and complex assemblies, enabling the construction of highly detailed hierarchical detector models.

GEANT4 allows users to define materials with precise physical properties, including elemental composition, density, and temperature. The toolkit also includes an extensive built-in material database covering common detector substances, such as silicon, iron, glass, and gaseous mixtures. Characteristics such as radiation and interaction length, excitation energy loss coefficients in the Bethe-Bloch formula[5], and shell correction

factors are computed from the elemental composition.

These features enable users to configure the detector construction in GEANT4 with a high degree of flexibility and accuracy. Accurate geometry and material definitions are essential for simulating energy deposition, scattering, absorption, and track propagation, all of which directly influence the detector response and data reconstruction. Overall, the detector construction module in GEANT4 forms the foundation for subsequent physics processes and plays a central role in the major simulation workflows.

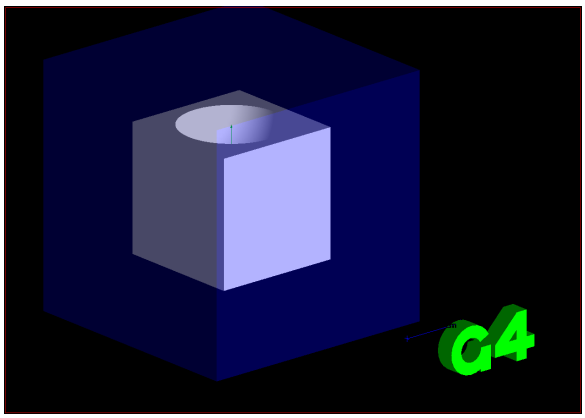


Figure 1: Compound solid formed by subtracting a hemisphere from a cube using `G4SubtractionSolid` in GEANT4.

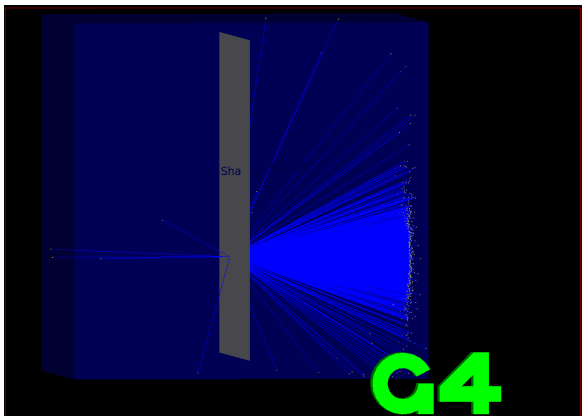


Figure 2: Simulation of alpha particle scattering by a thin Gold foil target in GEANT4.

2.3 Rutherford Experiment Simulation

The Rutherford scattering experiment, historically pivotal in establishing the nuclear model of the atom, can be effectively reproduced using GEANT4 to demonstrate the elastic scattering of alpha particles from a thin metallic foil. The core objective of this simulation was to visualize the angular deflection of alpha particles with respect to Geiger and Marsden's experimental observations.

A simulation of Rutherford scattering was performed, in which a beam of 20,000 alpha particles of energy 5.5 MeV was incident on a gold foil of thickness 2 μm (see Figure 2). It was observed that only 3 out of the 20,000 alpha particles were backscattered. This demonstrates the rare nature of large-angle scattering in Rutherford's experiment. This provides insights into the interaction between particles and matter. Geiger and Marsden had reported that roughly 1 in 8000 alpha particles were backscattered in their experiment[6], and the simulation result is consistent with this observation.

3 INO-ICAL Detector

The India-based Neutrino Observatory (INO) is a flagship experiment aimed at studying atmospheric neutrinos with high precision. Its primary detector, the Iron Calorimeter (ICAL)[7], is a tracker-type sampling calorimeter that uses alternating layers of magnetized iron plates and RPCs. RPCs act as active tracking elements that record the spatial information of traversing charged particles, particularly muons. This layered structure

enables the precise reconstruction of particle trajectories by capturing multiple position and timing hits as the particles pass through successive RPC layers.

The large mass and modular design allow for the accumulation of statistically significant event samples over a few years. In this study, we utilized a simulation-generated dataset based on the ICAL geometry and response to develop machine learning models for muon energy reconstruction.

3.1 Overall Geometry and Structure

The ICAL detector is designed to be a large modular structure with an overall dimension of approximately 48 m × 16 m × 14.5m, divided into three modules, each measuring approximately 16 m × 16m × 14.5 m. As shown in Figure 3, each module consists of a stack of 151 horizontal iron layers, each 5.6 cm thick, separated by a 4 cm air gap housing the RPCs.

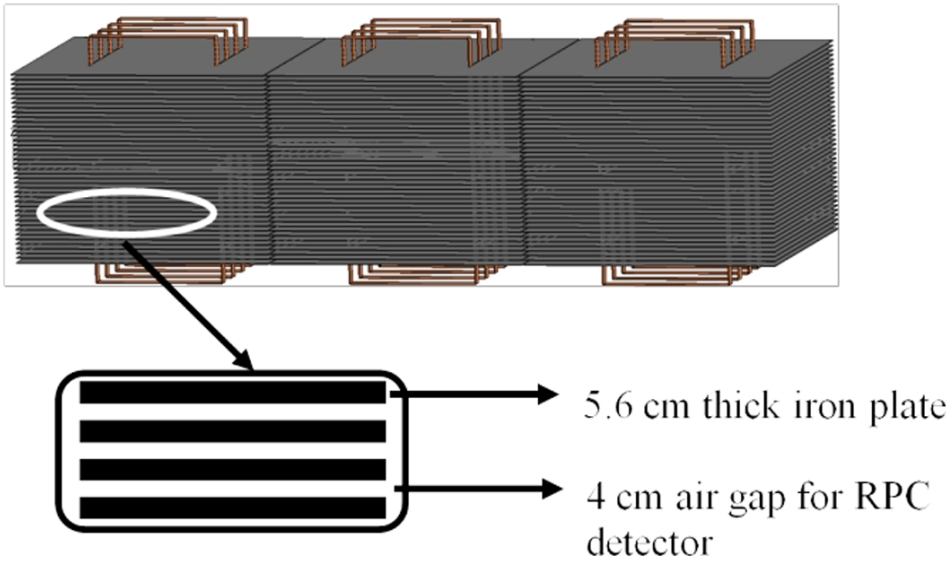


Figure 3: Schematic view of the 50 kt ICAL Detector.

Each module has two vertical slots for current-carrying copper coils that are wound around it. These generate a magnetic field of approximately 1.5 T in the central region of the module. The Iron plates serve two purposes: they act as a passive target material, inducing interactions of neutrinos, and they provide a medium for the magnetic field. The strong magnetic field in the central region allows the measurement of the curvature of the muon tracks, which helps in charge identification.

The ICAL Detector incorporates approximately 14,000 iron plates of 2 m × 4 m area and a thickness of 5.6 cm, 30,000 RPCs of 2 m × 2 m area, and nearly 4,000,000 readout channels. Together, these components are expected to enable precise measurements that advance the study of neutrino properties.

3.2 Resistive Plate Chambers

Resistive Plate Chambers (RPCs) are gaseous parallel-plate detectors that combine excellent time resolution, high detection efficiency, and cost-effectiveness. They consist of

two parallel electrodes made of materials with high volume resistivity, such as Bakelite or glass, separated by a thin gas gap[8]. These electrodes were externally coated with graphite, which was used to distribute the high voltage on the electrodes[9]. It uses orthogonal readout strips on both sides of the gas gap, separated from the graphite coating by an insulator, commonly Mylar, as shown in Figure 4.

The RPCs serve as active tracking elements in the ICAL Detector. Each gas-filled RPC is 2 m × 2 m in size and sandwiched between layers of iron plates. The RPCs consisted of two 3 mm-thick glass electrodes separated by a 2 mm gas gap. The gas gap was composed of Freon (R134A) (∼95%), isobutane, and trace amounts of SF_6. This combination of gas signals helps to keep the signal localized, and the location is used to reconstruct the trajectory of the charged particle in the detector. A high DC voltage was applied across the electrodes to establish a uniform electric field across the gas volume.

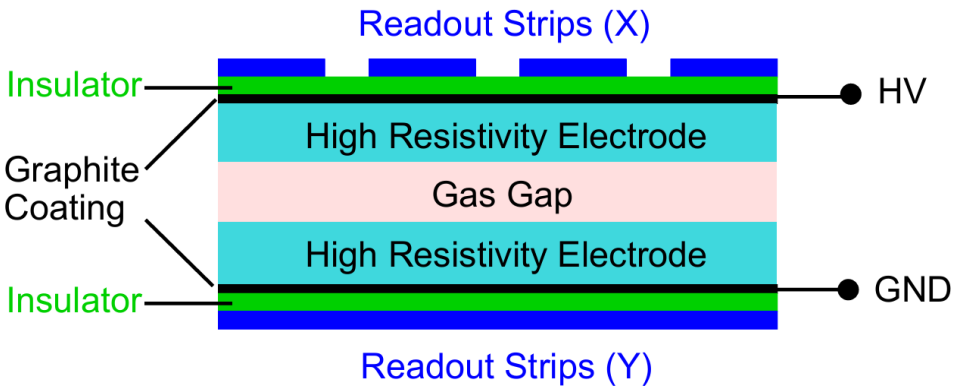


Figure 4: The internal structure of an RPC module.

When a muon (or any charged particle) passes through the gas gap, it ionizes the gas molecules and creates electron-ion pairs. These are accelerated in an electric field towards the electrodes. As the electrons gain kinetic energy from the field, they acquire sufficient energy to ionize additional gas molecules via secondary ionization. This leads to an exponential multiplication of electrons and positive ions in a process known as the Townsend avalanche[10]. Under certain conditions, the avalanche transitions into a streamer when the space charge from the accumulated ions distorts the electric field and enables a self-sustained discharge. Avalanche-mode RPCs suppress this by adding electronegative gases such as SF_6, which absorb free electrons and inhibit the formation of streamers. The avalanche induced a localized signal in the orthogonal pickup strips mounted externally. These 3 cm wide strips were arranged in the X and Y planes, providing a two-dimensional hit location for every interaction.

The generated signal is digital, marking the presence of a hit, and has a typical timing resolution of ∼1ns. This excellent time resolution allows the determination of the directionality of the muon, which is essential for distinguishing between neutrino and antineutrino events (upward-and downward-going muons).

The detector incorporates approximately 30,000 RPCs distributed across three modules. Each RPC is instrumented with dedicated front-end electronics and data acquisition systems to record the hits and timing information for every event.

3.3 Muon Tracking in ICAL

The INO-ICAL Detector identifies and reconstructs the trajectories of muons produced by atmospheric neutrino interactions. As a muon passes through the layered geometry, the RPCs interleaved between the iron plates register discrete hits at each layer by detecting the ionization of gas and the resulting induced signals from the pickup strips. The strip signals provide X and Y coordinates, whereas the layer number provides the Z coordinate. These measurements yield the 3-D position of the muon at each interaction point.

The multiple hits recorded across successive RPC layers are linked using reconstruction algorithms to form a continuous track. The granularity of the RPCs and uniform layer spacing allow for an accurate determination of the trajectory. The copper coils in each module produce a magnetic field of approximately 1.5T in the central region. The muon path bends owing to the Lorentz force. The curvature of the reconstructed track reveals both the momentum and charge of the muon—positive muons bend in one direction, while negative muons bend in the opposite direction.

Track reconstruction accounts for multiple scattering, detector resolution, and measurement uncertainties. Algorithms such as the Kalman Filter are employed to fit the track and extract kinematic parameters with high precision[7]. Tracks spanning several RPC layers, even at energies of a few GeV, can be reliably reconstructed. By combining spatial coordinates, timing information, and curvature, the detector determines the complete muon trajectory, its direction, momentum, and charge. This capability underpins INO-ICAL's ability to investigate atmospheric neutrino interactions and contributes directly to the precision measurements in neutrino physics.

4 Machine Learning

Machine learning (ML) has emerged as a transformative approach in high-energy physics (HEP), offering powerful tools for analyzing large volumes of complex, high-dimensional data produced in modern experiments. Traditional analysis methods often struggle with such data because of their limited scalability and flexibility. In contrast, ML techniques, including neural networks, boosted decision trees, graph neural networks, and autoencoders, enable the extraction of meaningful features with improved sensitivity and computational efficiency [11].

The utility of ML is evident in several key domains of particle physics. In event classification, ML algorithms separate signal events from background processes based on the patterns in the detector response. For particle identification, pattern recognition methods interpret complex detector responses to differentiate between particle types. ML also plays a growing role in anomaly detection, facilitating data-driven searches for rare and previously unmodeled phenomena.

In neutrino detection experiments, ML has been effectively used for tasks such as event selection, direction/distance reconstruction and calorimetric corrections. The use of supervised regression techniques and neural network models for muon energy reconstruction is particularly relevant, as ML learns to map observables (e.g., hit coordinates, curvature, and timing information) to muon energy more accurately and flexibly than analytic methods. The ability to incorporate complex correlations and nonlinear dependencies helps reduce resolution errors and bias, particularly in low-energy regimes, where conventional reconstruction methods struggle.

In summary, machine learning has become an integral component of modern particle physics, offering improvements in event classification, reconstruction, simulation, and data analysis. In this study, we explored the application of supervised ML models—specifically, Artificial Neural Networks (ANN), XGBoost, and Random Forest—for muon energy reconstruction using a simulation-generated dataset based on the INO-ICAL detector geometry and response. These models aim to capture the complex correlations among detector observables and provide accurate data-driven energy estimates that complement the conventional reconstruction techniques.

4.1 Dataset Description and Preprocessing

4.1.1 Dataset Overview

The dataset used in this study was provided by Mr. Arnab Sarkar and was originally generated using the GEANT4 simulation toolkit. Each instance in the dataset corresponds to a simulated particle interaction, represented by a six-dimensional vector consisting of five input features and one target variable. The features are as follows:

- **Number of Hits:** Total number of times the particle interacted with sensitive detector elements.
- **Zenith Angle (θ):** Incident angle of the particle with respect to the detector Z-axis, measured in radians.
- **Δx, Δy, Δz:** Displacement in the x, y and z-directions, respectively, between the initial and final hit (in mm).

  The target variable is:
- **Energy:** The true simulated energy of the particle (in GeV).

4.1.2 Feature Engineering

To enhance the representational capability of the input features, an additional variable, referred to as **Linear Displacement**, was engineered. This feature is calculated as the Euclidean distance between the initial and final hit positions of a particle within the detector volume:

$$\text{Linear Displacement} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{1}$$

Notably, this feature does not capture the actual path length traversed by the particle. Owing to the presence of a magnetic field in the detector, charged particles such as muons follow curved trajectories. In contrast, the above expression assumes a straight-line motion between the first and last recorded hits, and therefore only provides an approximate measure of the true trajectory.

Despite the simplification involved in its calculation, linear displacement provides a physically meaningful measure of a particle's traversal through the detector material. It is expected to show a strong positive correlation with the muon energy, making it a relevant input for regression-based predictions. Figure 6 illustrates this relationship, indicating the potential predictive value of the engineered features. Figure 5 presents the relationship between the initial energy and the zenith angle (θ), illustrating the angular dependence of particle energy across the dataset.

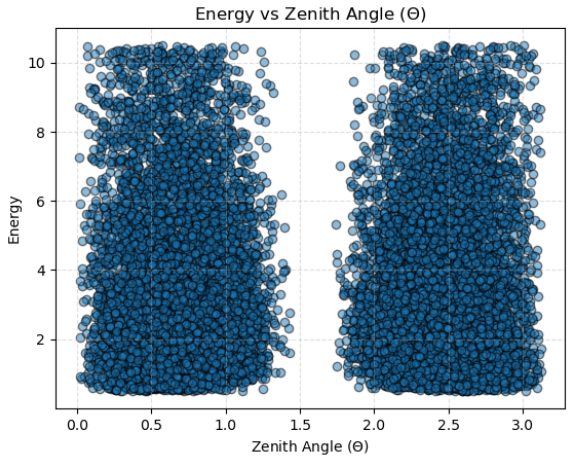


Figure 5: Scatter plot showing the correlation between muon energy and zenith angle (θ)

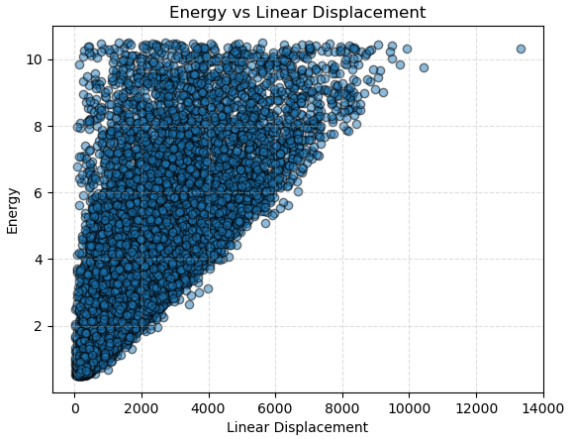


Figure 6: Scatter plot showing the correlation between muon energy and the engineered feature-Linear Displacement.

4.1.3 Data splitting and Normalization

The original dataset consisted of 13,800 events. We randomly divided the data into training and testing subsets using the `train_test_split` function from the Scikit-learn library[12]. The test set comprised 2,760 events, representing 20% of the total data, whereas the remaining 11,040 events (80%) were used to train the models.

Prior to model training, we applied min-max normalization to all input features. This transformation scales each feature x to a fixed range of [0, 1] using the following equation:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{2}$$

where $x_{\min}$ and $x_{\max}$ represent the minimum and maximum values of the feature in the training set, respectively. Normalization ensures that all features contribute proportionally during learning, which is particularly important for gradient-based models, such as Artificial Neural Networks. It also improves numerical stability and accelerates convergence by mitigating the disparities in the feature scale. We used normalized training data to train the models and reserved the test data to evaluate their performance.

4.2 Random Forest Regression Model

The Random Forest (RF) model employed in this study is a supervised ensemble learning algorithm specifically designed for regression tasks. This algorithm was originally introduced by Breiman [13]. It is grounded in the principle of the *wisdom of the crowd*, which posits that aggregating the predictions from multiple weak learners typically yields more accurate and robust results than relying on a single model [14]. In this approach, a collection of decision trees is trained independently, and the final prediction is obtained by averaging the outputs of all the trees in the ensemble. This method significantly reduces the variance of the predictions and enhances the generalization performance.

The structure of the Random Forest model is illustrated in Figure 7. The model configuration, training methodology, tree construction mechanism, and evaluation criteria are described in the following subsections.

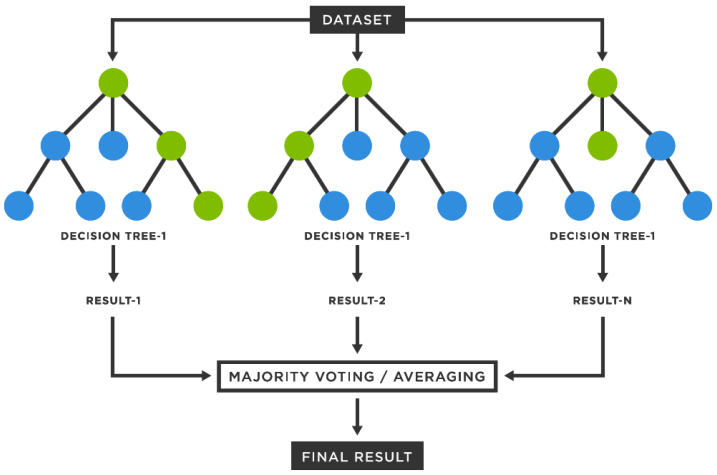


Figure 7: Illustration of the Random Forest model architecture. Source: Adapted from [15]

4.2.1 Model Configuration

The Random Forest Regressor was implemented using Scikit-learn [12] with the following hyperparameters: the number of estimators was set to 100, meaning that the ensemble comprised 100 independently trained decision trees. No maximum depth was specified, allowing each tree to grow fully until all leaves were pure or contained fewer than two samples. The minimum number of samples required to split an internal node was set to 2. Bootstrapping was enabled, meaning that each tree was trained on a random sample drawn with replacement from the training dataset.

4.2.2 Training and Prediction Mechanism

The model is trained on a dataset $\{(x_i, y_i)\}_{i=1}^{N}$, where each input $x_i \in \mathbb{R}^6$ represents a six-dimensional feature vector and $y_i \in \mathbb{R}$ denotes the corresponding scalar target.

The training process involves the following key steps.

1. For each of the $T = 100$ trees in the ensemble, the algorithm draws a bootstrap sample from the training data by sampling with replacement.
2. At each internal node of a tree, it selects the optimal feature and corresponding split point that maximizes variance reduction in the target variable.
3. The tree grows recursively by repeating the splitting process until it reaches a stopping condition. Specifically, a node is no longer split if it becomes pure (i.e., contains samples with identical target values) or contains fewer than two samples.

After training, the prediction for a new input x is computed by averaging the predictions of all the individual trees:

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^{T} f_t(x) \tag{3}$$

where $T = 100$ denotes the total number of trees and $f_t(x)$ is the prediction made by the tth decision tree.

4.2.3 Tree Construction: Splitting Criteria and Node Purity

At every decision node, the tree selects a feature and threshold value to split the dataset such that a certain criterion is optimized. For regression tasks, the most commonly used criterion is variance reduction, which aims to minimize the mean squared deviation of the target variable within each partition. The impurity measure I at a node is defined as:

$$I = \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2 \tag{4}$$

where $\bar{y}$ is the mean of the target values in the node and n is the number of samples. The algorithm evaluates all possible splits across randomly selected features and chooses the one that results in the greatest decrease in impurity.

4.2.4 Out-of-Bag Validation

The Out-of-Bag (OOB) score provides an internal cross-validation mechanism for Random Forests. Each tree is trained on a bootstrap sample, leaving out approximately one-third of the training data, which is then used to evaluate the tree. The aggregated OOB prediction serves as a reliable estimate of the model generalization performance. It was calculated using the coefficient of determination (R^2 score).

Given:

- y_i: true value of the ith data point,
- $\hat{y}_i$: predicted value of the ith data point,
- $\bar{y} = \frac{1}{N} \sum_{i=1}^{N} y_i$: mean of the true values,
- N: total number of samples,

The R^2 score is defined as

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2} \tag{5}$$

The numerator represents the residual sum of squares (RSS), and the denominator represents the total sum of squares (TSS). Thus, R^2 measures the fraction of the total variance in y captured by the predictions $\hat{y}$. A value of R^2 closer to 1 indicates a better fit, with the model capturing a larger fraction of the variance in the data.

The reported OOB score for this model was

$$\text{OOB Score} = \texttt{0.9061}$$

4.3 XGBoost Regression model

Extreme Gradient Boosting, or XGBoost, is a high-performance and scalable implementation of gradient-boosted decision trees. It builds an ensemble of weak learners in an additive manner using second-order optimization and regularization, thereby improving generalization, as shown in Figure 8. The notation used here is borrowed from the original XGBoost Manuscript [16].

4.3.1 Model Configuration

The `XGBRegressor` model was implemented using the `xgboost` library with the following hyperparameter settings:

- Number of trees (`n_estimators`): 500
- Maximum depth of trees (`max_depth`): 6
- Learning rate (η): 0.01
- Subsample ratio: 0.8 (without replacement)
- Feature sampling: 0.8

4.3.2 Model Formulation

Given an input $\mathbf{x}_i \in \mathbb{R}^m$, the XGBoost model predicts the output $\hat{y}_i \in \mathbb{R}$ as the sum of the outputs from an ensemble of K regression trees:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F} \tag{6}$$

Each $f_k \in \mathcal{F}$ is a function defined by a tree structure q and a set of leaf weights $w \in \mathbb{R}^T$ such that

$$f(\mathbf{x}) = w_{q(\mathbf{x})}, \quad q : \mathbb{R}^m \to \{1, \ldots, T\} \tag{7}$$

where T is the number of leaves in the tree and $q(\mathbf{x})$ assigns an input to its corresponding leaf index.

4.3.3 Objective Function and Optimization

The learning objective at iteration t is to minimize the regularized loss:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) + \Omega(f_t) \tag{8}$$

where l is a convex differentiable loss function and the regularization term is

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{9}$$

Here, γ penalizes the number of leaves and λ penalizes the ℓ_2-norm of the leaf weights.

To optimize $\mathcal{L}^{(t)}$, a second-order Taylor approximation is used around the current prediction:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{N} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) \tag{10}$$

where:

$$g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad \text{(first-order Gradient)}$$

$$h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}, \quad \text{(second-order Hessian)}$$

Let $I_j = \{i \mid q(\mathbf{x}_i) = j\}$ denote the set of instances assigned to leaf j. Then, the optimal leaf weight w_j^* and the corresponding optimal value of the approximated objective function are:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{11}$$

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^{T} \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \tag{12}$$

Equation 12 is used as a scoring function to measure the quality of the tree structures. Suppose I_L and I_R are instances of the left and right nodes after the split, and $I = I_L \cup I_R$, then the splitting criterion for the node is given by:

$$\mathcal{L}_{\text{split}} = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i \right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \tag{13}$$

The learning curve of the model was obtained by evaluating the MSE loss on the test set after each iteration, as shown in Figure 9.

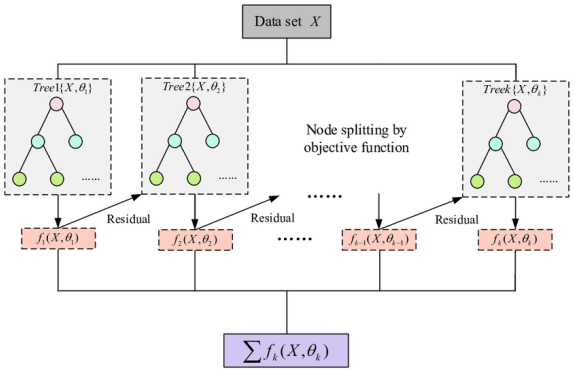


Figure 8: XGBoost Tree Ensemble Architecture. Source: Adapted from [17]

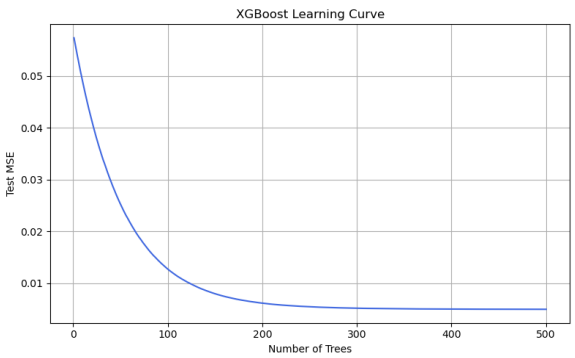


Figure 9: Learning Curve of the XGBoost Model

4.3.4 Feature Importance

Feature importance plots from the Random Forest and XGBoost Regressor models quantify the influence of each feature on energy prediction. Both models identified Δz as the most important feature, with the highest cumulative gain across the trees (see Figure 10 and 11). This aligns with the physical intuition that higher-energy muons travel longer distances, traverse more layers, and show greater displacement along the z-axis. Therefore, Δz has the highest correlation with energy.

The XGBoost Regressor showed relatively lower and more uniform feature importance values than the Random Forest. This results from its configuration: 500 estimators and 80% feature sampling per split to control the overfitting. Feature sampling distributes importance across features, thereby reducing individual dominance. Random Forest, trained with 100 estimators and no feature sampling, assigned more distinct importance values.

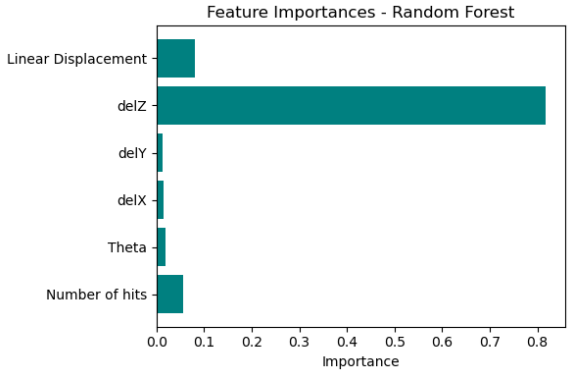


Figure 10: Feature importance from the Random Forest Regressor model.

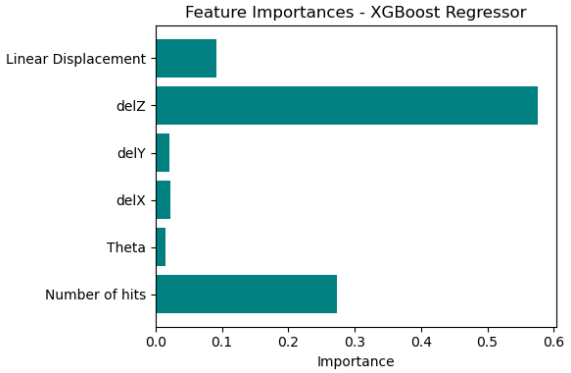


Figure 11: Feature importance from the XGBoost Regressor model.

4.4 Artificial Neural Network (ANN)

The Artificial Neural Network (ANN) employed in this study was a fully connected, feed-forward deep neural network designed specifically for regression tasks. The model was implemented using the TensorFlow library [18]. A schematic representation of the network architecture is presented in Figure 12. The architecture and training methodology are described in the following subsections.

4.4.1 Model Architecture

The architecture of the ANN model consisted of an input layer, four hidden layers, and an output layer. The detailed configuration is as follows.

1. **Input Layer (I):** 6 neurons
2. **Hidden Layer 1 ($\mathbf{h_1}$):** 256 neurons with ReLU activation
3. **Hidden Layer 2 ($\mathbf{h_2}$):** 128 neurons with ReLU activation
4. **Hidden Layer 3 ($\mathbf{h_3}$):** 64 neurons with ReLU activation
5. **Hidden Layer 4 ($\mathbf{h_4}$):** 32 neurons with ReLU activation
6. **Output Layer (O):** 1 neuron with linear activation

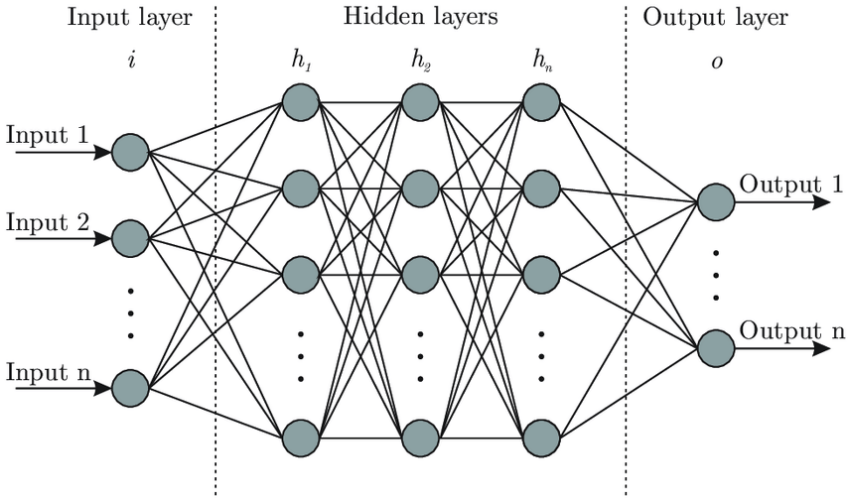


Figure 12: Schematic architecture of a fully connected feedforward artificial neural network (ANN). Source: Adapted from [19]

4.4.2 Forward Propagation

Given an input sample from the training set, denoted by $\mathbf{x} \in \mathbb{R}^6$, the network computes the output using the following sequence of operations:

$$\mathbf{h}_1 = \text{ReLU}(W_1\mathbf{x} + b_1) \tag{14}$$
$$\mathbf{h}_2 = \text{ReLU}(W_2\mathbf{h}_1 + b_2) \tag{15}$$
$$\mathbf{h}_3 = \text{ReLU}(W_3\mathbf{h}_2 + b_3) \tag{16}$$
$$\mathbf{h}_4 = \text{ReLU}(W_4\mathbf{h}_3 + b_4) \tag{17}$$
$$\hat{y} = W_o\mathbf{h}_4 + b_o \tag{18}$$

4.4.3 Backward Propagation

The network was trained to minimize the loss function using the Adam optimizer. Two models with different loss functions were implemented, one of them is Mean Squared Error(MSE) loss and other is Log-cosh loss [20]. Their definitions are as follows:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(f_\theta(x_i) - y_i\right)^2 \tag{19}$$

$$\mathcal{L}_{\text{log-cosh}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\log\left(\cosh\left(f_\theta(x_i) - y_i\right)\right) \tag{20}$$

where:

- θ: model parameters (i.e., all weights W and biases b across layers)
- x_i: the ith training sample
- y_i: the corresponding true target

- N: total number of training samples

The optimization was performed using the Adam algorithm [21, 22], a first-order gradient-based optimizer that computes adaptive learning rates for each of the parameters. It maintains moving averages of both the first and second moments of the gradients. Two auxiliary variables, $\mathbf{m}_t$ and $\mathbf{v}_t$, initialized to zero, are updated at each iteration t as follows:

$$g_t = \nabla_\theta \mathcal{L}_t(\theta_{t-1}) \tag{21}$$

where $\mathcal{L}_t(\theta_{t-1})$ is the objective function evaluated using the model parameters from a previous iteration.

The first and second moment estimates are computed as:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) g_t \tag{22}$$
$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) g_t^2 \tag{23}$$

with decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$. Because $\mathbf{m}_0$ and $\mathbf{v}_0$ are initialized to zero, they are biased towards zero. Therefore, bias correction is applied to mitigate early stage underestimation:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \tag{24}$$
$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \tag{25}$$

The model parameters were updated using the following rule:

$$\theta_{t+1} = \theta_t - \frac{\alpha \cdot \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{26}$$

where:

- $\alpha = 10^{-4}$: learning rate
- $\epsilon = 10^{-7}$: small constant to prevent division by zero

This process is repeated until the loss function converges to a minimum, yielding optimal model parameters.

4.4.4 Comparison of Learning Curves

The Log-Cosh loss exhibits a smoother and more stable learning curve than the MSE loss, which shows initial instability. This behavior can be explained by the asymptotic form of the log-cosh loss function [23].

For small deviations:

$$\log(\cosh(x)) \approx \frac{x^2}{2} \quad \text{as } x \to 0$$

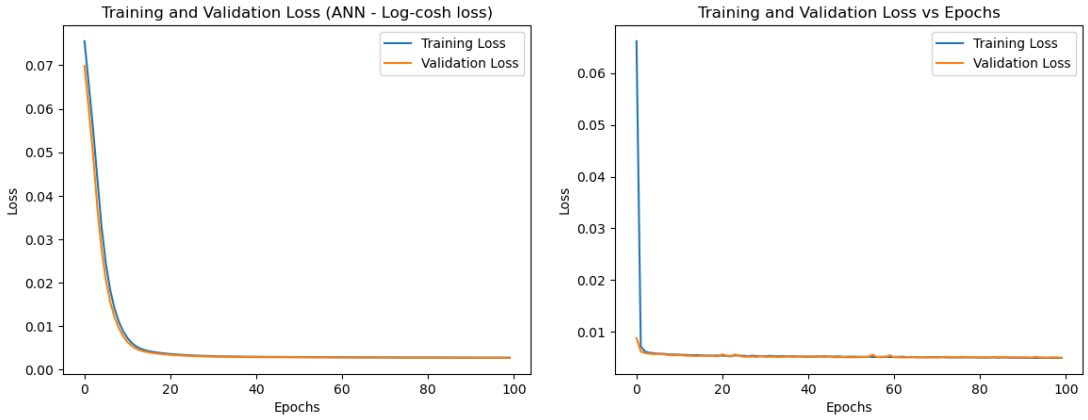


Figure 13: Learning curves of both ANN Models (ANN Log-cosh loss model) - Left and (ANN MSE Loss model) - Right

For large deviations:

$$\log(\cosh(x)) \approx |x| - \log(2) \quad \text{as } |x| \to \infty$$

Thus, Log-Cosh combines the sensitivity of MSE near zero with the robustness of MAE (see Equation 28) for the outliers. Its first derivative is bounded, ensuring controlled gradient updates with the ADAM optimizer and contributing to smoother convergence. However, the MSE loss function heavily penalizes the outliers, which causes an initial instability and a rough learning curve.

5 Model Evaluation and Comparison

This section evaluates the performance of four machine learning models: Random Forest Regressor (RF), XGBoost Regressor (XGB), and two Artificial Neural Networks trained with different loss functions—Mean Squared Error (ANN MSE Loss) and Log-cosh (ANN Log-cosh Loss). Each model was assessed using standard evaluation metrics that are widely adopted in machine learning. Additionally, we analyzed the energy residual (ΔE) distributions to gain deeper insights into the predictive accuracy. The final comparison summarizes the relative effectiveness and suitability of each model for the ICAL detector dataset.

5.1 Standard Evaluation Metrics

The performance of all four models was quantitatively compared using the following metrics implemented via the Scikit-learn library [12].

1. Mean Squared Error (MSE): according to the Equation 19
2. Root Mean Squared Error (RMSE):

$$\mathcal{L}_{\text{RMSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2} \tag{27}$$

3. Mean Absolute Error (MAE):

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i| \tag{28}$$

4. R^2 Score: as given in the Equation 5

The results are summarized in Table 1, which reports the evaluation scores for each model across all the metrics.

Model	**MSE**	**RMSE**	**MAE**	$\boldsymbol{R^2}$ **Score**
Random Forest	0.0053	0.0728	0.0476	0.9091
XGBoost	0.0049	0.0703	0.0465	0.9152
ANN – MSE Loss	0.0048	0.0695	0.0448	0.9173
ANN – Log-cosh Loss	0.0051	0.0716	0.0481	0.9122

Table 1: Performance metrics of all models on the test set.

5.2 Residual distribution evaluation

Although standard evaluation metrics offer a general overview of the model performance, they do not fully capture how well the predicted energies align with the true values. To address this limitation, we first constructed True vs. Predicted Energy plots for each model (Figures 14–17, left panels). The diagonal line ($y = x$) in each plot represents the ideal predictions, and deviations from this line indicate discrepancies between the predicted and actual energies.

To further examine the prediction errors, we plotted histograms of the energy residuals, defined as $\Delta E = E_{\text{predicted}} - E_{\text{true}}$. These distributions illustrate the frequency and spread of the errors, allowing for a comparative visual assessment of the model accuracy. To quantify the central tendency and variability of the residuals, we performed a Gaussian fit on the central portion of each histogram. This fitting was conducted using the `scipy.optimize.curve_fit` function from the SciPy library [24]. We selected the central region to include approximately 80% of the data, centered around the mode of the distribution. This approach intentionally excluded the outer 20% of the events to reduce the influence of outliers and to focus on the dominant predictive behavior of each model.

We evaluated the Gaussian fits of each model's residual distribution by comparing them to the ideal Dirac delta function centered at zero, which has a standard deviation of $\sigma = 0$. Among all models, the ANN trained with Log-cosh loss exhibited the narrowest distribution, with the smallest standard deviation $\sigma = 3.14 \times 10^{-2}$, indicating the closest resemblance to the Dirac delta function. Additionally, we compared the means of the Gaussian fits and observed that the ANN trained with MSE loss produced a distribution with a mean value closest to zero, with $\mu = 2.47 \times 10^{-3}$. A combined view of the residual distributions for all the models is shown in Figure 18.

The ANN models achieved slightly better performance than the tree-based models, possibly because of their ability to capture the complex nonlinear relationships between the input features and muon energy. Unlike tree ensembles, which rely on orthogonal partitions of the feature space, neural networks learn smooth and differentiable functions. This may have led to a lower residual bias and narrower error distributions.

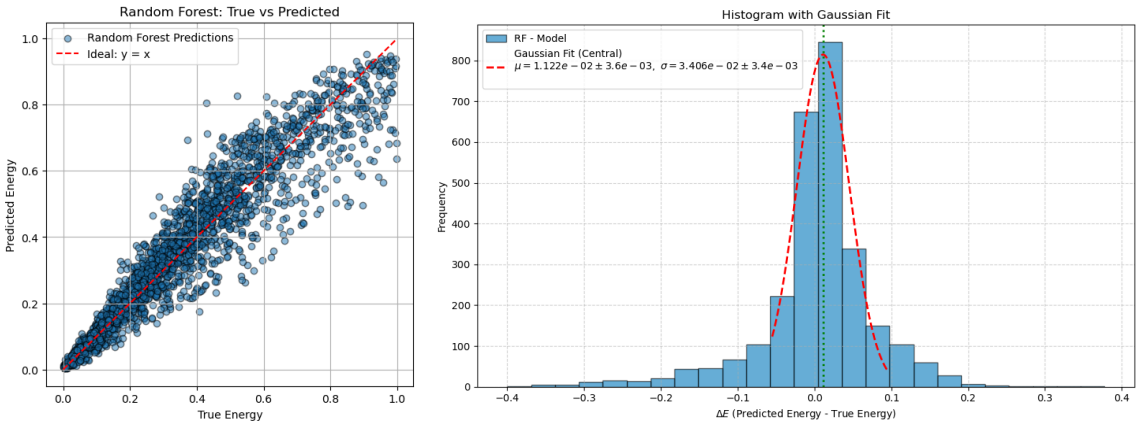


Figure 14: Random Forest Regressor: True vs. Predicted Muon Energy (left) and Gaussian fit to the residual distribution (right).

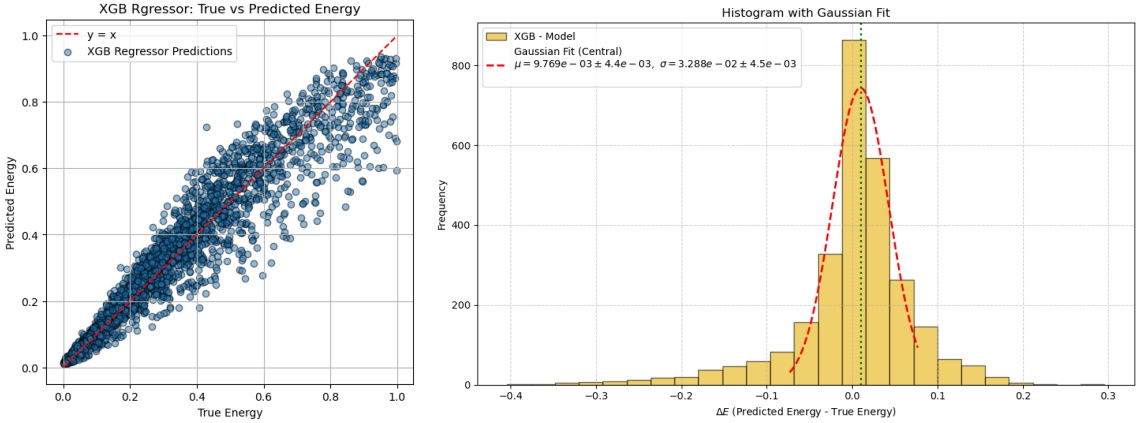


Figure 15: XGBoost Regressor: True vs. Predicted Muon Energy (left) and Gaussian fit to the residual distribution (right).

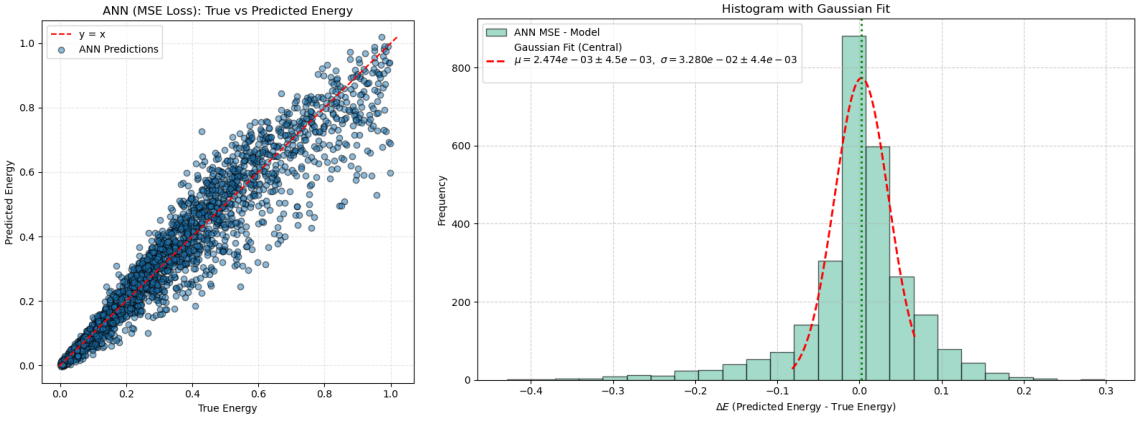


Figure 16: Artificial Neural Network (MSE Loss): True vs. Predicted Muon Energy (left) and Gaussian fit to the residual distribution (right).

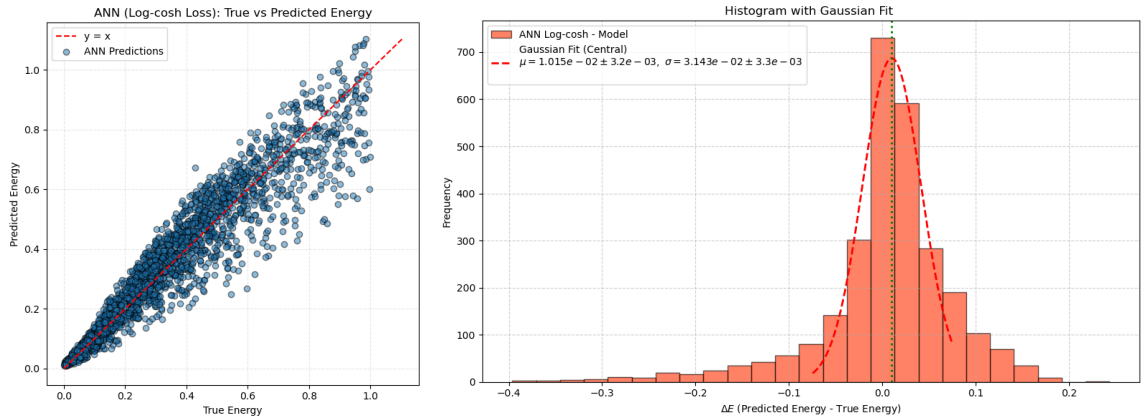


Figure 17: Artificial Neural Network (Log-cosh Loss): True vs. Predicted Muon Energy (left) and Gaussian fit to the residual distribution (right).

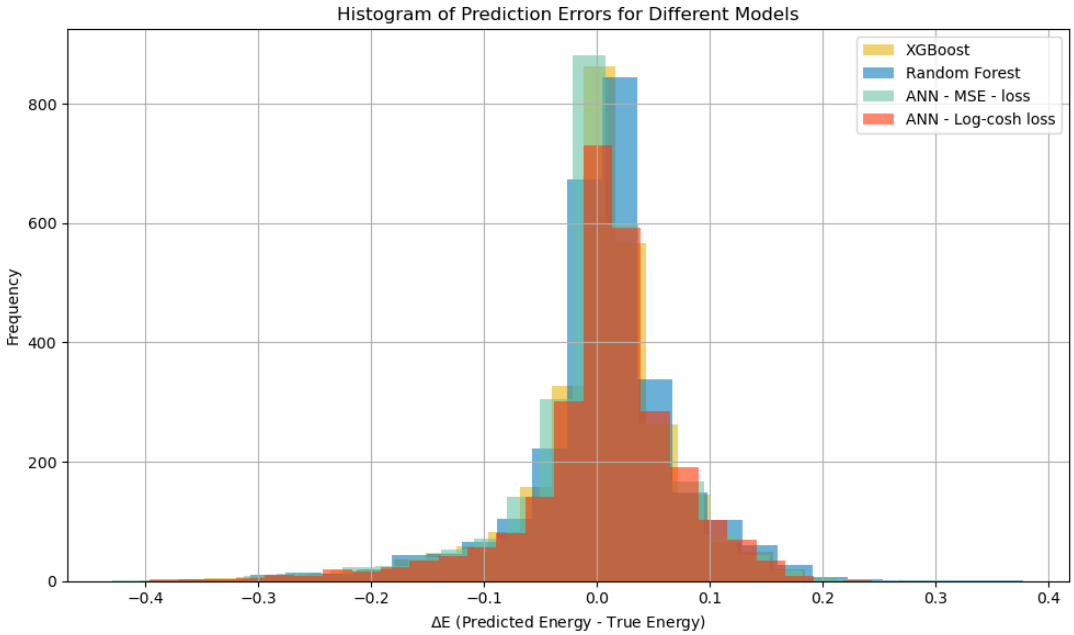


Figure 18: Comparison of residual histograms for all models. The distribution width and central peak indicate the relative prediction accuracy.

6 Conclusion

This study explored the application of machine learning techniques for muon energy reconstruction using data from the INO-ICAL detector. We began with a brief study of detector simulations using GEANT4, including the modeling of Rutherford scattering and the construction of detector geometries. The INO-ICAL detector design, based on magnetized iron layers and Resistive Plate Chambers (RPCs), was discussed in the context of muon tracking and energy reconstruction.

We implemented and evaluated four supervised regression models: Random Forest, XGBoost, and two Artificial Neural Networks (ANN) trained with MSE and Log-cosh loss functions. The underlying mechanisms and mathematical formulations of each model are

discussed in detail. Their performance was assessed using standard regression metrics and residual analysis. Although all models performed comparably, the ANN models exhibited slightly better performance in terms of a lower standard deviation and mean error in the residual distributions.

Overall, this study highlights the effectiveness of combining simulation-based datasets with machine learning methods to improve energy reconstruction in modern particle physics experiments.

7 Future Direction

This study demonstrates the use of data-driven techniques for tasks such as parameter reconstruction in neutrino detectors. Future studies may incorporate higher-dimensional datasets and larger sample volumes, enabling the development of more advanced ANN architectures capable of handling increased input complexity. Additionally, this study serves as a benchmark for comparing different regression models, which can guide future improvements in reconstruction accuracy and in model selection.

References

[1] Geant4 Collaboration. *Geant4: Book For Application Developers*. CERN, release 11.3 edition, 2025. Rev9.2 - June 14th.

[2] Tullio Basaglia, Zane W. Bell, Daniele D'Agostino, Paul V. Dressendorfer, Simone Giani, Maria Grazia Pia, and Paolo Saracco. Geant4: A game changer in high energy physics and related applicative fields. *Future Generation Computer Systems*, 159:411–422, 2024.

[3] GEANT Collaboration, S Agostinelli, et al. Geant4–a simulation toolkit. *Nucl. Instrum. Meth. A*, 506(25):0, 2003.

[4] Geant4 Collaboration. *Physics Reference Manual*. CERN, release 11.3 edition, 2024.

[5] William R. Leo. *Techniques for Nuclear and Particle Physics Experiments: A How-to Approach*. Springer-Verlag Berlin Heidelberg, Berlin, Heidelberg, 2nd edition, 1994.

[6] H Gegier and Ernest Marsden. On a diffuse reflection of the α-particles. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, 82(557):495–500, 1909.

[7] The ICAL Collaboration. Invited review: Physics potential of the ical detector at the india-based neutrino observatory (ino). *Pramana*, 88(5), April 2017.

[8] Cms rpc technical note. Technical report, CERN, 2002.

[9] Christian Lippmann. *Detector Physics of Resistive Plate Chambers*. Dissertation, Johann Wolfgang Goethe-University, Frankfurt am Main, Frankfurt am Main, Germany, 2003.

[10] R. R. Shinde and E. Yuvaraj. Fabrication and characterization of glass resistive plate chamber (rpc). Technical report, Tata Institute of Fundamental Research, 2019. Presented at the Experimental High Energy Physics School, TIFR, 2019.

[11] Biplob Bhattacherjee and Swagata Mukherjee. Modern machine learning and particle physics: an in-depth review. *European Physical Journal Special Topics*, 233:2421–2424, 2024. Published online 16 October 2024.

[12] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, and et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[13] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[14] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Sebastopol, CA, 2nd edition, 2019.

[15] Deniz Gunay. Random forest. `https://medium.com/@denizgunay/random-forest-af5bde5d7e1e`, 2023. Accessed: 2025-07-21.

[16] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. *arXiv preprint arXiv:1603.02754*, 2016.

[17] Fraidoon Omarzai. Xgboost classification in-depth. https://medium.com/@fraidoonomarzai99/xgboost-classification-in-depth-979f11ef4bf9, 2023. Accessed: 2025-07-21.

[18] Martín et al. Abadi. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from https://www.tensorflow.org/.

[19] Facundo Bre, Juan Gimenez, and Víctor Fachinotti. Prediction of wind pressure coefficients on building surfaces using artificial neural networks. *Energy and Buildings*, 158, 11 2017.

[20] Qi Wang, Yue Ma, Kun Zhao, and Yingjie Tian. A comprehensive survey of loss functions in machine learning. *Annals of Data Science*, 9(2):187–212, 2022.

[21] Diederik P Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR 2015)*, 2015.

[22] Cristian Leo. The math behind the adam optimizer. *Towards Data Science*, 2025. Accessed July 21, 2025.

[23] Resve A Saleh and AK Saleh. Statistical properties of the log-cosh loss function used in machine learning. *arXiv preprint arXiv:2208.04564*, 2022.

[24] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, and et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.